

STM32F411CEU6 Bare-Metal LED Blink with ST-Link

A from-scratch tutorial: startup code, linker script, GPIO registers, ST-Link flashing, and GDB debugging.

0. What this builds

This project blinks the onboard LED on an STM32F411CEU6 board, usually a Black Pill-style board with the LED on PC13. The workflow uses a custom linker script, a minimal startup file, direct RCC/GPIO register access, and ST-Link over SWD for flashing and debugging. No STM32CubeIDE, HAL, or generated project is required.

```
write C + startup + linker script
    |
    v
arm-none-eabi-gcc produces blink.elf
    |
    v
OpenOCD talks to ST-Link
    |
    v
ST-Link talks to STM32 over SWD
    |
    v
firmware is written to Flash at 0x08000000
    |
    v
CPU resets, reads vector table, calls Reset_Handler, reaches main()
    |
    v
RCC enables GPIOC, PC13 toggles, LED blinks
```

The STM32F411CE has a Cortex-M4 core, up to 512 KB Flash, 128 KB SRAM, and STM32F4 peripheral blocks documented in the datasheet and RM0383 reference manual. The exact board wiring is separate from the MCU; PC13 is a common LED pin but not guaranteed on every board.

1. Mental model before the files

Reset does not call main()

On a Cortex-M microcontroller, reset starts from a vector table, not from C runtime magic. The first word is the initial stack pointer. The second word is the address of Reset_Handler. Your firmware image must be laid out so the core sees those values at the beginning of Flash.

```
0x08000000: initial stack pointer, usually top of SRAM
0x08000004: Reset_Handler address
0x08000008: NMI_Handler address
0x0800000C: HardFault_Handler address
...
```

The linker script is part of the boot contract

The linker script tells the linker where Flash and RAM physically exist, where to put the vector table, where code lives, where initialized globals run, where zero-initialized memory goes, and where the stack starts. On a desktop OS, the loader handles this. On bare metal, you do.

GPIO is memory-mapped I/O

A line like `GPIOC_MODER |= (1U << 26)` is a bus write to a hardware register, not normal RAM. That register controls transistor networks inside the GPIO peripheral. The chain is C write -> CPU store -> bus transaction -> peripheral register update -> pin driver changes -> physical voltage on the pin changes.

RCC clock enable comes before GPIO configuration

STM32 peripherals are clock-gated. GPIOC exists in the address map, but the peripheral logic must be clocked before it behaves normally. That is why the first hardware step is enabling the GPIOC clock in RCC_AHB1ENR.

ST-Link replaces DFU for flashing and adds debugging

With ST-Link, you do not need BOOT0 or USB DFU. ST-Link talks to the chip over SWD. OpenOCD can program Flash, halt/reset the CPU, expose a GDB server, set breakpoints, single-step, and inspect registers/memory.

2. Toolchain and wiring

Install tools

```
# Debian/Ubuntu
sudo apt install gcc-arm-none-eabi make openocd gdb-multiarch

# macOS
brew install arm-none-eabi-gcc openocd
```

ST-Link wiring

ST-Link pin	STM32 board pin	Purpose
SWDIO	SWDIO	bidirectional debug data
SWCLK	SWCLK	debug clock
GND	GND	shared reference
3.3V / VREF	3V3 / VREF	target voltage reference; do not assume it powers the board well

Powering the board from USB-C while ST-Link supplies SWDIO/SWCLK/GND/VREF is usually safest. The critical mistake is forgetting common ground.

3. Project layout

```
stm32f411-blink/
|-- Makefile
|-- linker.ld
`-- src/
    |-- startup.c
    -- main.c
```

4. linker.ld

This file defines physical memory and places sections. It is small, but it is one of the most important files in the project.

```
ENTRY(Reset_Handler)

MEMORY
{
    FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 512K
    RAM (rwx) : ORIGIN = 0x20000000, LENGTH = 128K
}

_estack = ORIGIN(RAM) + LENGTH(RAM);

SECTIONS
{
    .isr_vector :
    {
        KEEP(*( .isr_vector))
    } > FLASH
```

```

.text :
{
    *(.text*)
    *(.rodata*)
    . = ALIGN(4);
    _etext = .;
} > FLASH

.data :
{
    . = ALIGN(4);
    _sdata = .;
    *(.data*)
    . = ALIGN(4);
    _edata = .;
} > RAM AT > FLASH

_sidata = LOADADDR(.data);

.bss :
{
    . = ALIGN(4);
    _sbss = .;
    *(.bss*)
    *(COMMON)
    . = ALIGN(4);
    _ebss = .;
} > RAM
}

```

- ENTRY names the reset symbol for the linker.
- FLASH and RAM describe the physical memory map.
- _estack is the initial stack pointer value.
- .isr_vector must stay at the beginning of Flash.
- .data runs in RAM but loads from Flash, so startup code must copy it.
- .bss lives in RAM and must be zeroed before main().

5. src/startup.c

The startup file creates the vector table and performs the minimal C runtime initialization before main().

```

#include <stdint.h>

extern uint32_t _estack;

extern uint32_t _sidata;
extern uint32_t _sdata;
extern uint32_t _edata;

extern uint32_t _sbss;
extern uint32_t _ebss;

int main(void);

void Reset_Handler(void);
void Default_Handler(void);

void NMI_Handler(void)    __attribute__((weak, alias("Default_Handler")));
void HardFault_Handler(void) __attribute__((weak, alias("Default_Handler")));

__attribute__((section(".isr_vector")))
const void *vector_table[] = {
    &_estack,
    Reset_Handler,
    NMI_Handler,
    HardFault_Handler,
};

void Reset_Handler(void)

```

```

{
    uint32_t *src = &_sidata;
    uint32_t *dst = &_sdata;

    while (dst < &_edata) {
        *dst++ = *src++;
    }

    dst = &_sbss;
    while (dst < &_ebss) {
        *dst++ = 0;
    }

    main();

    while (1) {
    }
}

void Default_Handler(void)
{
    while (1) {
    }
}

```

The vector table is data consumed by the CPU. Reset_Handler then copies initialized globals, zeros BSS, and calls main. If main ever returns, the CPU loops forever instead of wandering into random memory.

6. src/main.c

This version directly manipulates RCC and GPIOC registers. It assumes a PC13 LED, usually active-low on Black Pill-style boards.

```

#include <stdint.h>

#define AHB1PERIPH_BASE    0x40020000UL

#define GPIOC_BASE        (AHB1PERIPH_BASE + 0x0800UL)
#define RCC_BASE          (AHB1PERIPH_BASE + 0x3800UL)

#define RCC_AHB1ENR       (*(volatile uint32_t *) (RCC_BASE + 0x30UL))

#define GPIOC_MODER        (*(volatile uint32_t *) (GPIOC_BASE + 0x00UL))
#define GPIOC_OTYPER       (*(volatile uint32_t *) (GPIOC_BASE + 0x04UL))
#define GPIOC_OSPEEDR      (*(volatile uint32_t *) (GPIOC_BASE + 0x08UL))
#define GPIOC_PUPDR        (*(volatile uint32_t *) (GPIOC_BASE + 0x0CUL))
#define GPIOC_BSRR         (*(volatile uint32_t *) (GPIOC_BASE + 0x18UL))

#define RCC_AHB1ENR_GPIOCEN (1U << 2)
#define LED_PIN             13U

static void delay(volatile uint32_t count)
{
    while (count--) {
        __asm volatile ("nop");
    }
}

int main(void)
{
    RCC_AHB1ENR |= RCC_AHB1ENR_GPIOCEN;

    GPIOC_MODER &= ~(3U << (LED_PIN * 2U));
    GPIOC_MODER |= (1U << (LED_PIN * 2U));

    GPIOC_OTYPER &= ~(1U << LED_PIN);
    GPIOC_OSPEEDR &= ~(3U << (LED_PIN * 2U));
    GPIOC_PUPDR &= ~(3U << (LED_PIN * 2U));

    while (1) {
        GPIOC_BSRR = (1U << (LED_PIN + 16U)); // PC13 low
        delay(800000);
    }
}

```

```

        GPIOC_BSRR = (1U << LED_PIN);          // PC13 high
        delay(800000);
    }
}

```

- RCC_AHB1ENR bit 2 enables GPIOC clock.
- MODER uses two bits per pin; 01 means general-purpose output.
- OTYPER = 0 selects push-pull.
- OSPEEDR low speed is enough for an LED.
- PUPDR = 00 disables internal pull-up/pull-down.
- BSRR lower 16 bits set pins; upper 16 bits reset pins.

7. Makefile

```

TARGET = blink

CC      = arm-none-eabi-gcc
OBJCOPY = arm-none-eabi-objcopy
SIZE    = arm-none-eabi-size

CFLAGS = -mcpu=cortex-m4 \
        -mthumb \
        -mfloat-abi=hard \
        -mfpu=fpv4-sp-d16 \
        -Wall \
        -Wextra \
        -O0 \
        -g \
        -ffreestanding \
        -fdata-sections \
        -ffunction-sections \
        -nostdlib

LDFLAGS = -T linker.ld \
        -Wl,-Map=$(TARGET).map \
        -Wl,--gc-sections \
        -nostdlib

SRCS = src/startup.c src/main.c
OBJS = $(SRCS:.c=.o)

all: $(TARGET).elf $(TARGET).bin

$(TARGET).elf: $(OBJS)
■$(CC) $(CFLAGS) $(OBJS) $(LDFLAGS) -o $@
■$(SIZE) $@

$(TARGET).bin: $(TARGET).elf
■$(OBJCOPY) -O binary $< $@

clean:
■rm -f $(OBJS) $(TARGET).elf $(TARGET).bin $(TARGET).map

```

Build with make. The ELF is the primary artifact for ST-Link/OpenOCD because it contains machine code, addresses, symbols, and debug information. The BIN is a raw image, useful for simple flashing tools but less useful for debugging.

8. Build and inspect

```

make
arm-none-eabi-objdump -h blink.elf
arm-none-eabi-objdump -d blink.elf
less blink.map

```

Look for `.isr_vector` and `.text` near `0x08000000`, and `.data/.bss` near `0x20000000`. This confirms the linker script is doing what you think it is doing.

9. Flash with ST-Link

```
openocd -f interface/stlink.cfg -f target/stm32f4x.cfg \  
-c "program blink.elf verify reset exit"
```

- interface/stlink.cfg tells OpenOCD to use ST-Link.
- target/stm32f4x.cfg selects the STM32F4 target family.
- program blink.elf writes the firmware using ELF section addresses.
- verify reads back Flash to check it programmed correctly.
- reset resets the target after programming.
- exit closes OpenOCD.

10. Debug with GDB

Terminal 1:

```
openocd -f interface/stlink.cfg -f target/stm32f4x.cfg
```

Terminal 2:

```
gdb-multiarch blink.elf
```

```
(gdb) target extended-remote localhost:3333  
(gdb) monitor reset halt  
(gdb) break main  
(gdb) continue
```

After it stops at main, single-step and inspect the actual peripheral registers:

```
(gdb) next  
(gdb) x/wx 0x40023830 # RCC_AHB1ENR  
(gdb) x/wx 0x40020800 # GPIOC_MODER  
(gdb) x/wx 0x40020818 # GPIOC_BSRR  
(gdb) info registers  
(gdb) disassemble
```

This is the learning advantage of ST-Link: you can prove that the RCC bit changed, prove that MODER changed, and catch faults instead of only watching the LED.

11. Common failure modes

- OpenOCD sees ST-Link but not the target: check SWDIO, SWCLK, GND, target power, and VREF.
- OpenOCD only works with sudo on Linux: fix udev rules later; sudo confirms it is a permissions issue.
- Flash succeeds but LED does not blink: verify the LED pin and active-low/active-high wiring.
- Code appears stuck: break at HardFault_Handler or Default_Handler and inspect PC/LR/xPSR.
- Sections are at wrong addresses: the linker script or flags are wrong.
- A fake delay loop is not accurate timing; replace it with SysTick/timers later.

12. What to read next

1. STM32F411xC/xE datasheet: <https://www.st.com/resource/en/datasheet/stm32f411ce.pdf>
2. RM0383 STM32F411xC/E reference manual:
https://www.st.com/resource/en/reference_manual/rm0383-stm32f411xce-advanced-armbased-32bit-mcus-stmicroelectronics.pdf
3. PM0214 STM32 Cortex-M4 programming manual: https://www.st.com/resource/en/programming_manual/pm0214-stm32-cortexm4-mcus-and-mpus-programming-manual-stmicroelectronics.pdf

4. OpenOCD user guide: <https://openocd.org/doc/pdf/openocd.pdf>

Use the datasheet for chip limits and package facts, RM0383 for STM32F411 peripheral registers, PM0214 for Cortex-M4 reset/exception behavior, and OpenOCD docs for flashing/debugging commands.

Final mental model

```
ST-Link writes ELF sections to Flash
CPU resets
CPU reads vector table at 0x08000000
CPU loads stack pointer
CPU jumps to Reset_Handler
Reset_Handler initializes RAM
main() enables GPIOC clock
main() configures PC13 as output
main() writes BSRR to drive PC13 low/high
LED turns on/off because current flows or stops flowing
```